



An-Najah National University
Faculty of Engineering and IT

جامعة النجاح الوطنية
كلية الهندسة وتكنولوجيا المعلومات

Distributed Operating Systems

10636456

2nd Semester | 2025-2026

Bazar-com Project | Part 1

Roaa Bzoor	12218040
Yaman Abo Asal	12240055

Bazar.com is a small multi-tier online bookstore designed using a **microservices-based architecture**. The system follows a two-tier structure consisting of a **frontend service** and a **backend tier**, which includes a catalog service and an order service. All services communicate using RESTful HTTP APIs.

The main objective of this project is to demonstrate the principles of distributed systems by dividing the application into independent services that communicate over the network.

The system is implemented using **Python** and **Flask**.

The application manages a fixed set of four books, where each book has an ID, title, topic, price, and quantity in stock. Each service runs independently and communicates with other services using HTTP requests.

The system consists of three microservices:

- Frontend Service
- Catalog Service
- Order Service

1. Frontend Service (Port 5000)

The frontend service acts as the entry point for client requests. It provides REST endpoints such as:

- `/search/: topic`
- `/info/:id`
- `/purchase/:id`

It forwards requests to the backend services (Catalog and Order services) using HTTP requests.

2. Catalog Service (Port 5001)

The catalog service is responsible for managing the book inventory. It stores data in a CSV file (`catalog/catalog.csv`), which contains all book information including ID, title, topic, price, and quantity.

It provides the following endpoints:

- `GET /search/: topic` → returns books by topic
- `GET /info/:id` → returns details of a specific book
- `POST /update/:id` → updates book quantity or price

3. Order Service (Port 5002)

The order service handles purchase requests and order processing. It communicates with the catalog service to verify stock availability and update inventory accordingly.

It provides the following endpoint:

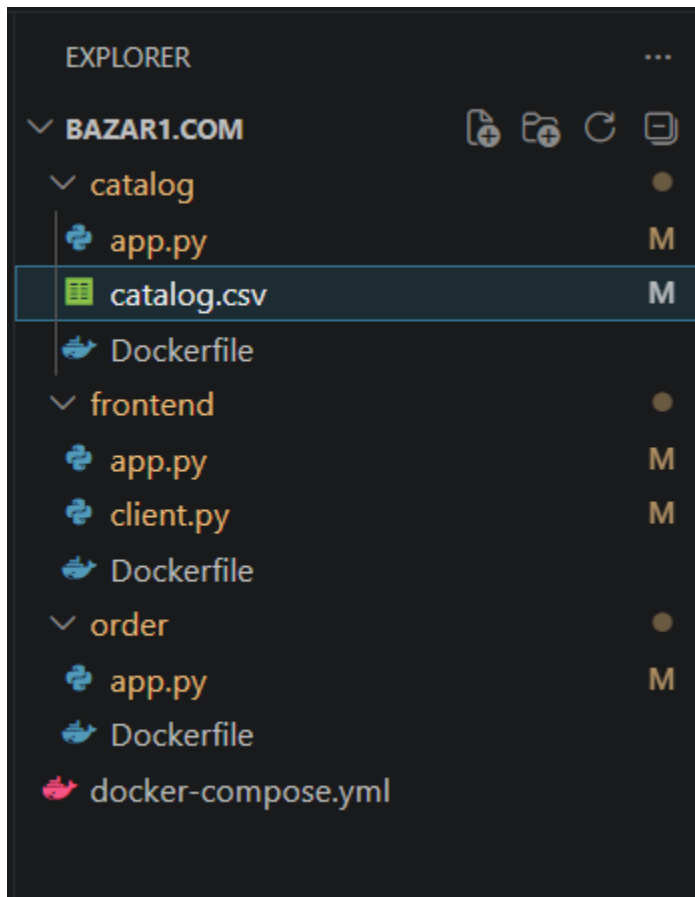
- `POST /purchase/:id` → processes purchase requests

The catalog service uses a CSV file (`catalog/catalog.csv`) to persist book data. This allows the system to maintain data even after restarting the service.

The order service manages order information during runtime.

Each service runs independently on a separate port using Flask. The system is containerized using Docker, and docker-compose is used to run all services together in a distributed environment.

The following photo shows the hierarchy of the system files:



Now, for the components of each service and how it works:

1. Catalog Service

The Catalog Service is responsible for managing the book inventory and providing information about available books. The data is stored in a CSV file (catalog/catalog.csv), which contains all book details such as ID, title, topic, price, and quantity.

```
3
4 # Create Flask application instance
5 app = Flask(__name__)
6
7 # Define the CSV file that stores the catalog data
8 FILE = "catalog.csv"
```

```
# Search books by topic
@app.route('/search/<topic>')
def search(topic):
    data = read_catalog()
    result = []
    # Filter books by topic
    for book in data:
        if book['topic'].lower() == topic.lower():
            result.append({"id": book['id'], "title": book['title']})
    return jsonify(result)
```

```
# Get information about a specific book by ID
@app.route('/info/<int:id>')
def info(id):
    data = read_catalog()

    # Search for the book by ID
    for book in data:
        if int(book['id']) == id:
            return jsonify(book)
    return jsonify({"error": "not found"}), 404
```

The result for “http://127.0.0.1:5001/search/distributed%20systems”

GET Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 33 ms • 278 B • ...

{ } JSON Preview Visualize

```
1 {
2   {
3     "id": "1",
4     "title": "How to get a good grade in DOS in 40 minutes a day."
5   },
6   {
7     "id": "2",
8     "title": "RPCs for Noobs."
9   }
10 }
11
```

The result for “http://127.0.0.1:5001/info/1”

GET Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 6 ms • 297 B • ...

{ } JSON Preview Visualize

```
1 {
2   "id": "1",
3   "price": "20",
4   "quantity": "9",
5   "title": "How to get a good grade in DOS in 40 minutes a day.",
6   "topic": "distributed systems"
7 }
8
```

The result for "http://127.0.0.1:5001/update/1"

Body Cookies Headers 5 Test Results 200 OK • 13 ms • 1:

{ } JSON Preview Visualize

```
1 {
2   "message": "updated"
3 }
4
```

GET http://127.0.0.1:5001/info/1 Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 6 ms • 297 B • ...

{ } JSON Preview Visualize

```
1 {
2   "id": "1",
3   "price": "20",
4   "quantity": "8",
5   "title": "How to get a good grade in DOS in 40 minutes a day.",
6   "topic": "distributed systems"
7 }
8
```

GET /books

```
69 # Get all books
70 @app.route('/books')
71 def books():
72     data = read_catalog()
73     return jsonify(data)
74
```

The result for “http://127.0.0.1:5001/books”

The screenshot shows a web browser interface with the address bar displaying `http://127.0.0.1:5001/books`. The browser status bar indicates a successful GET request with a 200 OK status, 43 ms response time, and 484 B of data. The main content area displays a JSON array of book objects. A yellow arrow points from the 'books' part of the URL in the address bar to the 'id' field of the first book object in the JSON response.

```
1 [
2   {
3     "id": "1",
4     "price": "20",
5     "quantity": "8",
6     "title": "How to get a good grade in DOS in 40 minutes a day.",
7     "topic": "distributed systems"
8   },
9   {
10    "id": "2",
11    "price": "50",
12    "quantity": "2",
13    "title": "RPCs for Noobs.",
14    "topic": "distributed systems"
15  },
16  {
17    "id": "3",
18    "price": "30",
19    "quantity": "15",
20    "title": "Xen and the Art of Surviving Undergraduate School.",
21    "topic": "undergraduate school"
22  },
23  {
24    "id": "4",
25    "price": "25",
26    "quantity": "8",
27    "title": "Cooking for the Impatient Undergrad.",
28    "topic": "undergraduate school"
29  }
30 ]
```

2. Order Service

The Order Service is responsible for handling purchase requests and processing orders. The service communicates with the Catalog Service using REST API calls to verify stock availability and update the inventory after a successful purchase.

The service is implemented using Python and Flask and runs on **port 5002**.

```
28
29 if __name__ == "__main__":
30     app.run(host="0.0.0.0", port=5002)
```

The service connects to the Catalog Service through REST requests:

- **GET /info/:id** → retrieves information about a specific book.
- **POST /update/:id** → decreases the book quantity after purchase.

```
6
7 CATALOG_URL = "http://127.0.0.1:5001"
8
```

Operations

- POST /purchase/:id

Steps

1. Read the book ID from the request URL.
2. Send a request to the Catalog Service using /info/:id to check the current quantity and book information.
3. If the quantity is equal to zero, return an "out of stock" message.
4. Otherwise, send a request to /update/:id to decrease the quantity by one.
5. Return a success message indicating that the book was purchased successfully.

The Order Service ensures that every purchase request:

- checks book availability,
- updates the catalog inventory,
- and processes the purchase correctly.

❖ GET /orders

```
55
56 # Get all orders
57 @app.route("/orders")
58 def get_orders():
59     return jsonify(orders)
60
```

The result of "http://127.0.0.1:5002/orders"

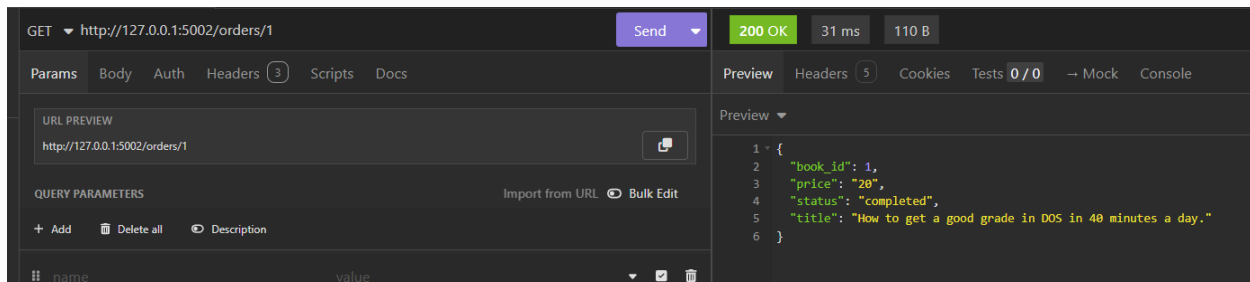
The screenshot shows a web browser interface with the URL `http://127.0.0.1:5002/orders` in the address bar. The status bar indicates a **200 OK** response with a response time of **16 ms** and a body size of **295 B**. The response body is displayed as a JSON array of three book orders:

```
[
  {
    "book_id": 1,
    "price": "20",
    "status": "completed",
    "title": "How to get a good grade in DOS in 40 minutes a day."
  },
  {
    "book_id": 2,
    "price": "50",
    "status": "completed",
    "title": "RPCs for Noobs."
  },
  {
    "book_id": 3,
    "price": "30",
    "status": "completed",
    "title": "Xen and the Art of Surviving Undergraduate School."
  }
]
```


❖ GET /orders/:id

```
61 # Get order by ID
62 @app.route('/orders/<int:id>')
63 def get_order(id):
64
65     for order in orders:
66         if order['book_id'] == id:
67             return jsonify(order)
68
69     return jsonify({"error": "order not found"}), 404
70
```

The result of “http://127.0.0.1:5002/orders/1”



The screenshot shows a web browser interface with the URL `http://127.0.0.1:5002/orders/1` in the address bar. The status bar indicates a `200 OK` response with a response time of `31 ms` and a body size of `110 B`. The browser's developer tools are open, showing the `Preview` tab. The response body is a JSON object:

```
1 {
2   "book_id": 1,
3   "price": "20",
4   "status": "completed",
5   "title": "How to get a good grade in DOS in 40 minutes a day."
6 }
```

3. Frontend Service

The Frontend Service acts as the main entry point for clients. It hides the internal communication between the Catalog Service and the Order Service and provides a simple REST API for users.

The service is implemented using Python and Flask and runs on **port 5000**.

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

The frontend communicates with:

- the Catalog Service on port 5001
- the Order Service on port 5002

```
CATALOG_URL = "http://catalog:5001"  
ORDER_URL   = "http://order:5002"
```

Operations:

- **GET /search/:topic**

```
2  
3 @app.route('/search/<topic>')  
4 def search(topic):  
5     resp = requests.get(f"{CATALOG_URL}/search/{topic}")  
6     return jsonify(resp.json())  
7
```

The frontend receives a topic from the client, sends a request to the Catalog Service using:

```
GET /search/:topic
```

and then returns the JSON response to the client.

- **GET /info/:id**

```
@app.route('/info/<int:id>')  
def info(id):  
    resp = requests.get(f"{CATALOG_URL}/info/{id}")  
    return jsonify(resp.json())
```

The frontend sends a request to the Catalog Service using:

```
GET /info/:id
```

If the book exists, the frontend returns the book information to the client. Otherwise, it returns a 404 error.

- **POST /purchase/:id**

```
@app.route('/purchase/<int:id>', methods=['POST'])
def purchase(id):
    resp = requests.post(f"{ORDER_URL}/purchase/{id}")
    return resp.json()
```

The frontend forwards the purchase request to the Order Service using:

POST /purchase/:id

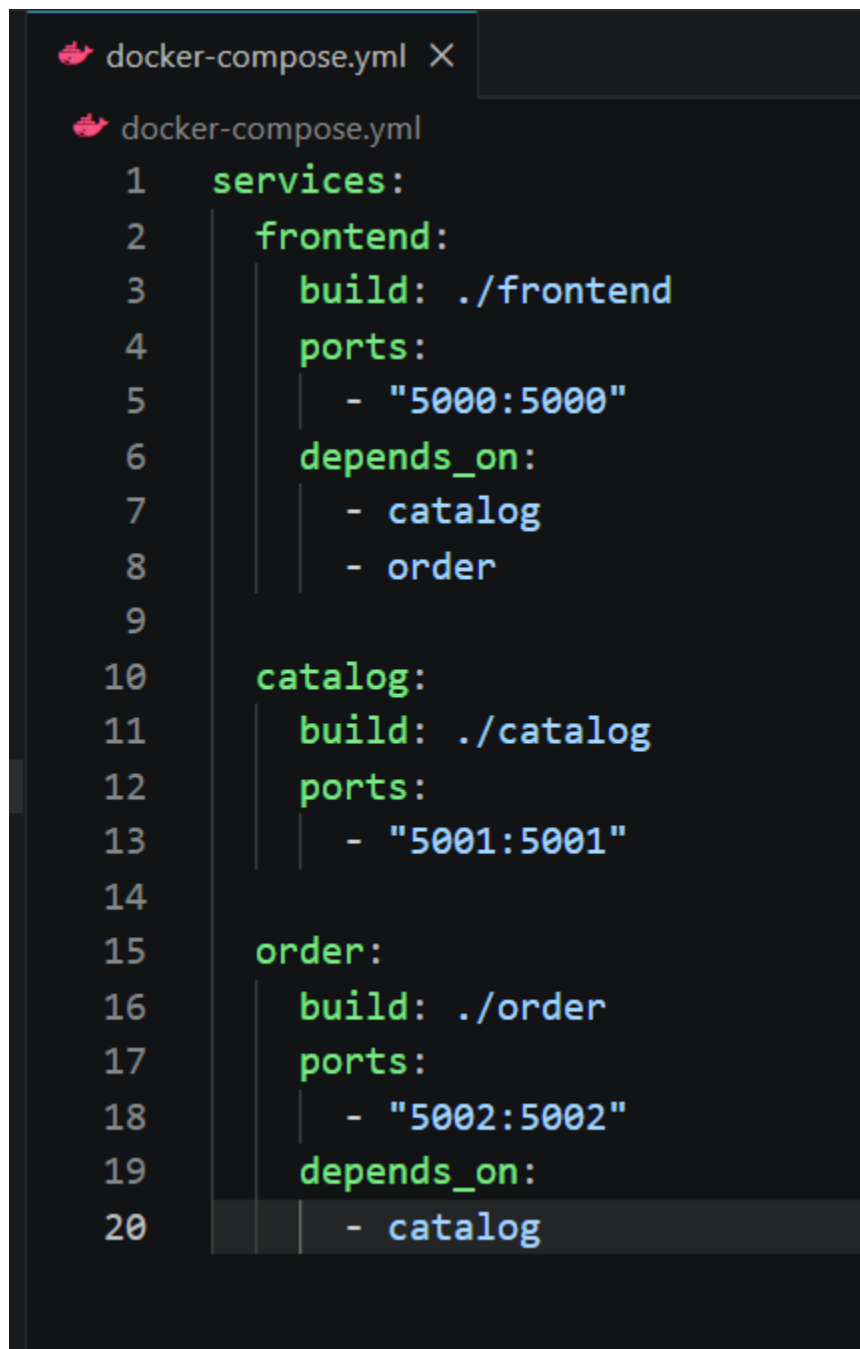
The response returned by the Order Service is then forwarded back to the client, including success or out-of-stock messages.

Docker Deployment

The project was containerized using Docker to ensure that each microservice runs in an isolated and consistent environment. Each service in the system (**catalog**, **order**, and **frontend**) has its own **Dockerfile** that defines how the service is built and executed inside a container.

All three services are orchestrated using a **docker-compose.yml** file located in the project directory. Docker Compose is used to manage and run the entire system with a single command, automatically building and starting each microservice from its respective folder.

Based on the configuration in the **docker-compose.yml** file, Docker Compose builds and runs each microservice (catalog, order, and frontend) in separate containers. Each service is started using its own **Dockerfile**. The file also defines the communication between services by specifying their dependencies. The frontend service depends on both the catalog and order services, while the order service depends on the catalog service. This ensures that the backend services are fully running before the frontend starts, enabling proper communication between all components of the system through REST APIs.



```
1  services:
2    frontend:
3      build: ./frontend
4      ports:
5        - "5000:5000"
6      depends_on:
7        - catalog
8        - order
9
10   catalog:
11     build: ./catalog
12     ports:
13       - "5001:5001"
14
15   order:
16     build: ./order
17     ports:
18       - "5002:5002"
19     depends_on:
20       - catalog
```

How to run my project using docker:

- 1- Open docker desktop.
- 2- Open the terminal on docker folder.
- 3- Build and start containers => docker-compose up --build